Evidence-Grounded Clinical-Trial Eligibility Screening with a LoRA-Tuned Qwen2.5 Model and Hand-Implemented KV-Cache, Tiled-Attention and Speculative Decoding

Imama Sarwar
Roll No. 23i-3048
Generative AI, Assignment 2
BSCS , FAST-NUCES Islamabad
i233048@isb.nu.edu.pk
GitHub Repo Link: https://github.com/ImamaSarwar/trial-eligibility-llm/tree/main
Hugging Face Link: https://huggingface.co/ISarwar/trial-lora

***Abstract*—Matching a patient to the eligibility criteria of a clinical trial is slow and error-prone when done manually. We build a criterion-level, evidence-grounded screening system on the public TrialGPT Criterion Annotations: an open-weight 1.5B-parameter decoder (Qwen2.5-1.5B-Instruct) is fine-tuned with LoRA to emit an expert-style eligibility label together with the IDs of supporting patient sentences. On a fixed patient-grouped split (37/8/8 patients, seed 42), fine-tuning raises held-out test accuracy from 7.3% to 71.8% (macro-F1 0.023 $\rightarrow$ 0.517) and evidence micro-F1 from 0.177 to 0.489. The dominant failure is that the model never answers "not enough information" for an exclusion criterion (0 of 28 on test) and treats missing information as a negative. We further implement from scratch (i) KV-cached greedy decoding that is token-identical to an uncached reference and up to $6.2\times$ faster at 1,024 prompt tokens on a T4 GPU, (ii) tiled online-softmax attention that matches reference attention to $\leq 10^{-6}$ and uses $10.9\times$ less peak memory at 2,048 tokens but is 8–44$\times$ slower than PyTorch's fused kernel, and (iii) greedy speculative decoding that is token-identical to target-only greedy decoding but 7–51% slower in our setting, which we explain with a simple cost model. All components are exposed in a local browser application with a live benchmark view and exportable reports. The system is an educational prototype and not a clinical enrollment tool.**

***Index Terms*—clinical trial matching, large language models, LoRA, KV cache, online softmax attention, speculative decoding, evidence retrieval**

I. Introduction

Recruiting patients into clinical trials requires comparing a free-text patient description with a list of inclusion and exclusion criteria, each of which can be satisfied, violated, not applicable, or simply undecidable from the note. Large language models can read both texts, but a useful screening aid must (a) keep the dataset's distinction between a negative decision and *insufficient information*, (b) point to the exact sentences it relied on, and (c) be small and fast enough to run on freely available hardware.

This work addresses six tasks: fine-tuning an open-weight Transformer on the TrialGPT Criterion Annotations [1], [2]; criterion-level label and evidence evaluation; a hand-written KV-cached decoder; a tiled online-softmax attention; a greedy speculative decoder; and an integrated browser application. Our contributions are:

- A reproducible fine-tuning and evaluation pipeline on a fixed patient-grouped split with a published manifest and no leakage of GPT-4 outputs into inputs.
- An honest evaluation that reports every label class, inclusion versus exclusion separately, and evidence retrieval with an explicit empty-set rule, including a detailed failure analysis.
- From-scratch implementations of KV-cached decoding, tiled online-softmax attention and greedy speculative decoding, each verified against a reference and benchmarked with warm-up and synchronized repeats.
- A local Gradio application that returns, for every criterion, a label, cited sentence IDs, the exact evidence text, and a rule-based preliminary outcome, with a downloadable report and a live decoder comparison.

Source code, split manifest, benchmark logs and the adapter are released (Appendix B).

II. Related Work

Trial matching with LLMs. TrialGPT [1] uses GPT-4 to retrieve trials, predict criterion-level eligibility with supporting sentences and explanations, and rank trials. The expert annotations released with it [2] are the benchmark used here; its GPT-4 predictions are deliberately excluded from our inputs. Unlike TrialGPT, we fine-tune a small locally executable model rather than calling a proprietary API.

Efficient adaptation. LoRA [3] freezes the pretrained weights and learns low-rank updates, which makes fine-tuning a 1.5B model feasible on a 16 GB GPU; we use the PEFT library [4]. We did not compare against full fine-tuning because of memory and time limits.

TABLE I
SPLIT SIZES. EMPTY-EVIDENCE = RECORDS WHOSE EXPERT EVIDENCE SET IS EMPTY.

Split	Patients	Annot.	Incl.	Excl.	Empty-evid.
Train	37	603	215	388	349
Val	8	206	66	140	116
Test	8	206	75	131	145

TABLE II
EXPERT LABEL COUNTS PER SPLIT.

Type	Label	Train	Val	Test
Incl.	included	88	35	27
	not included	13	3	7
	not enough information	109	27	38
	not applicable	5	1	3
Excl.	excluded	8	2	0
	not excluded	292	101	91
	not enough information	65	26	28
	not applicable	23	11	12

Architecture. The Qwen2.5 family [5] is a decoder-only Transformer [6] using rotary position embeddings [7] and grouped-query attention [8]; the latter keeps the KV cache small, which matters for our cache analysis.

Efficient attention. FlashAttention [9] computes exact attention in tiles without materializing the score matrix, building on the online normalizer calculation for softmax [10]. We re-implement the algorithm in plain PyTorch for correctness, not speed.

Efficient decoding. Speculative decoding [11] and speculative sampling [12] let a cheap draft model propose tokens that a large target model verifies in one pass, preserving the target's output distribution (and, under greedy decoding, its exact output).

III. MATERIALS AND METHODS

A. Dataset, split and leakage control

We use only `ncbi/TrialGPT-Criterion-Annotations`: 1,015 annotations over 53 patients. Each record has a numbered patient note, a trial and criterion (type and text), an expert eligibility label, and expert evidence sentence IDs. The inclusion label set is {included, not included, not enough information, not applicable}; the exclusion set is {excluded, not excluded, not enough information, not applicable}. The columns `gpt4_explanation`, `explanation_correctness`, `gpt4_sentences` and `gpt4_eligibility` are dropped before any processing, and the authors' own `training` flag is ignored.

Following the assignment, unique patient IDs are sorted, shuffled with `numpy.random.default_rng(42)`, and the first 37 / next 8 / last 8 patients form train / validation / test. A script asserts that the three patient sets are disjoint, and the manifest (`data/split_manifest.json`) lists every patient ID. Table I gives the resulting sizes and Table II the label distributions. The test split contains no "excluded" example, so that class can never be scored on test; "not included" has only 7 test examples.

Table II shows a strong imbalance that shapes the results: 75% of exclusion training labels are "not excluded" (292/388), and always predicting "not excluded" would score 44.2% accuracy on test (91/206).

B. Input/output format, tokenization and truncation

Each example is rendered with the model's chat template. The user turn contains the numbered patient note, the criterion type and text, the allowed labels for that type with a one-line meaning of "not enough information" and "not applicable", and an instruction to answer in JSON. The assistant turn (training target) is

```
{"label": "included",
 "evidence_sentences": [0, 1]}<|im_end|>
```

where the IDs are the dataset's original sentence numbers. Evidence text is never generated: it is looked up from the note by ID, so it cannot be invented. A tolerant parser extracts the JSON; a label that is not valid for the criterion type is counted as `invalid` (parse failure).

The maximum prompt length is set to 1,536 tokens. If exceeded, sentences are dropped from the end of the note and the dropped IDs are recorded, together with whether any gold evidence was lost. In practice nothing was truncated: the longest training example is 610 tokens (mean 316, 95th percentile 482) and the longest validation example 444 tokens; 0 of 603 training and 0 of 206 validation examples were truncated and no gold evidence was lost. For the test set ($n = 206$), 0 examples were truncated (`n_truncated` = 0).

C. Model selection and feasibility

We required an open-weight, locally executable decoder with a smaller same-tokenizer sibling for speculative decoding, a permissive license, and weights that fit a free 16 GB T4 for training. Table III compares the candidates. Memory is the fp16 weight size (2 bytes per parameter).

We chose Qwen2.5-1.5B-Instruct because (i) it fits comfortably on a T4 in fp16 for LoRA training and in fp32 on a 16 GB laptop (6.2 GB), (ii) it has an Apache-2.0 license with no gated download, which simplifies reproducibility, (iii) a 0.5B sibling with an identical tokenizer exists for speculative decoding, and (iv) grouped-query attention with only 2 KV heads keeps its KV cache small. The alternatives were rejected for practical reasons (gated, non-Apache licenses; a different tokenizer pair; larger memory for Gemma) and were *not* trained or compared experimentally owing to the time budget; this is a limitation. The KV cache of the target costs $2\,N_\ell N_{kv} d_h b = 2 \cdot 28 \cdot 2 \cdot 128 \cdot 2 = 28{,}672$ bytes per token (28 KiB), i.e. only about 33 MB for 1,152 tokens.

TABLE III
CANDIDATE DECODER MODELS.

Model	Params	Layers	Q/KV heads	Context	Vocab	License	fp16 weights	Decision
Qwen2.5-1.5B-Instruct	1.54B	28	12/2	32,768	151,936	Apache-2.0	3.09 GB	**target**
Qwen2.5-0.5B-Instruct	0.49B	24	14/2	32,768	151,936	Apache-2.0	0.99 GB	**draft**
Llama-3.2-1B-Instruct	1.24B	16	32/8	131,072	128,256	Llama 3.2 community	≈2.5 GB	considered, not used
Gemma-2-2B-it	2.6B	26	8/4	8,192	256,000	Gemma terms	≈5.2 GB	considered, not used

TABLE IV
FINE-TUNING CONFIGURATION.

LoRA rank / alpha / dropout	16 / 32 / 0.05
Trainable parameters	18,464,768 (1.18% of 1.56B)
Optimizer	AdamW, weight decay 0
Learning rate	2×10^{-4}, cosine, 5% warm-up
Epochs / optimizer steps	3 / 228
Batch size	1×8 gradient accumulation
Precision	fp16 autocast + loss scaling
Gradient clipping	1.0
Gradient checkpointing	yes
Seed	42
Wall-clock time	≈15 min on a Tesla T4

D. Fine-tuning

We train LoRA adapters on all attention and MLP projections (`q,k,v,o,gate,up,down`) of the fp16 base model, with adapters kept in fp32. The objective is the causal language-modelling cross-entropy restricted to the target tokens T (the prompt tokens are masked):

$$\mathcal{L} = -\frac{1}{|T|} \sum_{t \in T} \log p_\theta(y_t \mid y_{<t}, x). \tag{1}$$

Table IV lists the configuration. Hyperparameters were set once from common defaults and were *not* searched; the only selection made on validation data is the checkpoint (epoch) with the lowest validation loss.

Memory. fp16 weights take 3.09 GB; the adapter (18.5M fp32 parameters = 74 MB, matching the 73.9 MB adapter file), its gradients and the two AdamW moments add about 0.3 GB; activations are small because sequences are at most 610 tokens and gradient checkpointing is on. This fits the 15.6 GB T4 with a wide margin.

E. KV-cached autoregressive decoding

Reference. `generate_no_cache` re-runs the whole sequence at every step (`use_cache=False`) and takes the argmax at the last position. **Cached.** `generate_with_cache` performs one *prefill* over the prompt, keeps the returned cache, and then feeds *only the newest token* together with (i) the cache, (ii) an explicit `position_ids` equal to the current cache length, and (iii) an attention mask of ones of length cache+1. We do not rely on the library's `generate`.

Positional handling. Qwen2.5 uses rotary embeddings [7]: a key stored in the cache already carries the rotation of its absolute position, so a new token must be rotated by position L (the number of cached tokens), not 0. *Masking.* With batch size 1 and no padding the mask is all ones; causality for multi-token inputs (used by the verifier) is applied by the model relative to the cache length. *Cache growth.* The cache grows by one position per layer per generated token (28 KiB per token for the target). *Numerical differences.* Cached and uncached passes compute the same mathematics with different matrix shapes and reduction orders; in fp16 this can change low-order bits and, rarely, flip an argmax between two near-tied logits. We therefore test equality in fp32 on tiny random models and in fp16 on the real model.

Algorithm 1 Tiled attention with online softmax

for each query block Q_i **do**
  $m \leftarrow -\infty,\ \ell \leftarrow 0,\ o \leftarrow 0$
  for each key block (K_j, V_j) **do**
    if causal **and** block j entirely in the future **then break**
    end if
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; mask future entries to $-\infty$
    $m' \leftarrow \max(m, \mathrm{rowmax}(S))$; $\alpha \leftarrow e^{m-m'}$
    $P \leftarrow e^{S-m'}$; $\ell \leftarrow \alpha\ell + \mathrm{rowsum}(P)$
    $o \leftarrow \alpha o + PV_j$; $m \leftarrow m'$
  end for
  $O_i \leftarrow o/\ell$
end for

F. Tiled attention with online softmax

Reference attention computes $\mathrm{softmax}(QK^\top/\sqrt{d})V$ after subtracting the row maximum for stability and masking future keys with $-\infty$. The tiled version never forms the $N_q \times N_k$ matrix. For each query block it sweeps over key blocks while maintaining, per row, a running maximum m, normalizer ℓ and unnormalized output o:

$$m' = \max\big(m,\ \mathrm{rowmax}(S)\big), \quad \alpha = e^{m-m'}, \tag{2}$$

$$\ell' = \alpha\,\ell + \mathrm{rowsum}\big(e^{S-m'}\big), \tag{3}$$

$$o' = \alpha\,o + e^{S-m'}V_j, \qquad O = o/\ell, \tag{4}$$

where S is the scaled score tile. For causal masking, key blocks that lie entirely in the future of a query block are skipped, entries inside a block are masked with $-\infty$, and an infinite m' is replaced by 0 to avoid $\infty - \infty$ (Algorithm 1). Causality uses bottom-right alignment so the same code serves cached decoding ($N_q < N_k$).

G. Greedy speculative decoding

A 0.5B draft proposes k tokens greedily; the 1.5B target scores the pending token and all drafts in *one* forward pass and yields its own greedy choice $g_0, \ldots, g_k$ at every position.

Algorithm 2 Greedy speculative decoding round

state: caches C_t, C_d hold $L-1$ tokens; *pending* token
draft $d_1..d_k$ greedily from *pending* using C_d
one target pass on $[pending, d_1..d_k] \Rightarrow g_0..g_k$
$n \leftarrow$ largest $n \leq k$ with $g_i = d_{i+1}$ for all $i < n$
commit $d_1..d_n,\ g_n$; *pending*$\leftarrow g_n$
crop C_t to $L+n$; **if** $n = k$ feed d_k to draft **else** crop C_d to $L+n$

The longest prefix of drafts equal to $g_0, \ldots$ is accepted, and the target's next token (g_n) is appended: a correction on a mismatch, or a free bonus token if all k drafts match. The output is therefore identical to target-only greedy decoding. The invariant that makes this correct is: with L committed tokens, *both caches hold the first* $L-1$ *tokens* and the last committed token is pending. After each round both caches are cropped to the new length; if all drafts were accepted, the draft cache is one token short and that token is fed once (Algorithm 2). Draft and target share the same tokenizer vocabulary (checked at start-up).

H. Application workflow

Fig. 1 shows the system. The Gradio application (`python app/app.py`) runs locally on CPU in fp32 (about 8.2 GB of RAM for both models), loads the base model, merges the published LoRA adapter, and exposes three hand-written decoding modes. The user pastes a patient note (plain text or numbered sentences) and enters inclusion and exclusion criteria one per line. Inputs are validated (non-empty note, at most 6,000 characters, at least one and at most 12 criteria) and failures are shown as readable messages. For every criterion the application shows the label, cited sentence IDs, the evidence text copied from the note, and a templated explanation built only from the label and the cited sentences (the model itself outputs only label and IDs). Hallucinated sentence IDs are discarded.

The preliminary outcome is an explicit aggregation, not a labelled ground truth: (1) any unparsable prediction $\Rightarrow$ needs review; (2) any exclusion criterion "excluded" $\Rightarrow$ likely ineligible; (3) any inclusion criterion "not included" $\Rightarrow$ likely ineligible; (4) any "not enough information" $\Rightarrow$ needs review; (5) otherwise likely eligible, with a caution for each exclusion criterion marked "not excluded" without cited evidence, because absence of a mention is not proof of absence; "not applicable" is ignored. Every page carries the notice that this is an educational screening prototype and not a clinical enrollment decision. A benchmark tab runs the three decoders live on the current input and loads the logged GPU benchmark, and each screening can be downloaded as an HTML report with model, adapter, mode and timestamp.

Hardware and software. Training, evaluation and GPU benchmarks ran on a free Google Colab Tesla T4 (15.6 GB) using PyTorch 2.14.1+cu130, Transformers 5.22.1, PEFT 0.21.2, and Gradio 6.31.0. The application ran on a Dell Inspiron 14 5430 (Intel Core i7-1360P, 16 GB RAM, no CUDA GPU) in fp32 using PyTorch 2.15.0, Transformers 5.22.2, PEFT 0.21.2, and Gradio 6.31.0. Libraries: PyTorch [14], Transformers [13], PEFT, scikit-learn.

Evaluation decoding. Greedy decoding (`do_sample=False`), at most 64 new tokens, repetition penalty explicitly 1.0, fp16, batch size 8 with left padding; the same prompt and settings are used for the base and fine-tuned model. The test split was evaluated once for each model after all decisions had been made on validation data.

Classification metrics. Accuracy, per-class precision/recall/F1, confusion matrices and macro-F1, overall and separately for inclusion and exclusion. Macro-F1 averages over classes that occur in the gold labels or the predictions. **Evidence metrics.** Predicted and gold sentence-ID sets are compared per record. *Empty-set rule:* both empty $\Rightarrow$ P=R=F1=1; exactly one empty $\Rightarrow$ 0; otherwise standard set P/R/F1. We report (a) micro (pooled TP/FP/FN, empty/empty adds nothing), (b) macro over all records, and (c) macro over records with non-empty gold evidence. Because 145 of 206 test records have empty gold evidence, (b) is inflated by the rule and we treat (a) and (c) as the main numbers.

Benchmark protocol. Greedy decoding with end-of-sequence disabled so all modes produce exactly the same number of tokens, input text taken from validation notes cut to an exact token length, one warm-up run, five timed repeats with `cuda.synchronize()`, peak memory from `max_memory_allocated`, target with merged adapter in fp16, draft base model, $k = 4$.

IV. Results and Discussion

A. Training

Fig. 2 shows the training and validation loss. Training loss falls from 0.63 (step 5) to 0.035 (step 225). Validation loss is 0.1299, 0.1161 and 0.1232 after epochs 1–3: it rises in epoch 3 while training loss keeps falling, a sign of mild overfitting with only 37 training patients, so the epoch-2 checkpoint was kept.

B. Before/after fine-tuning

Table V compares the base model (zero-shot, identical prompt and decoding) with the fine-tuned model. The base model almost always answers "not applicable" (198 of 206 validation predictions), so accuracy is near zero. Fine-tuning gives $+68.5$ and $+64.5$ accuracy points on validation and test, with macro-F1 rising from 0.017/0.023 to 0.541/0.517 and evidence micro-F1 from 0.147/0.177 to 0.520/0.489. Validation and test accuracy agree closely (0.738 vs 0.718), so the validation set was not over-fitted. Context: always predicting "not excluded" would reach 49.0% (validation) and 44.2% (test) accuracy, so the gain is real but less dramatic than the baseline suggests, and macro-F1 (0.517) is a better summary than accuracy.

Table VI separates inclusion and exclusion criteria. Accuracy is similar for both (0.720 vs 0.718 on test), but macro-F1 is low for both because rare classes are missed, and exclusion evidence is far weaker (test micro-F1 0.118) than inclusion evidence (0.575): for exclusion criteria the model almost never

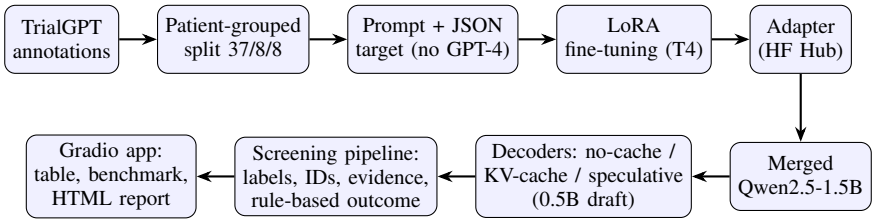


Fig. 1. System overview: offline training on Colab (top row) and local inference in the browser application (bottom row).

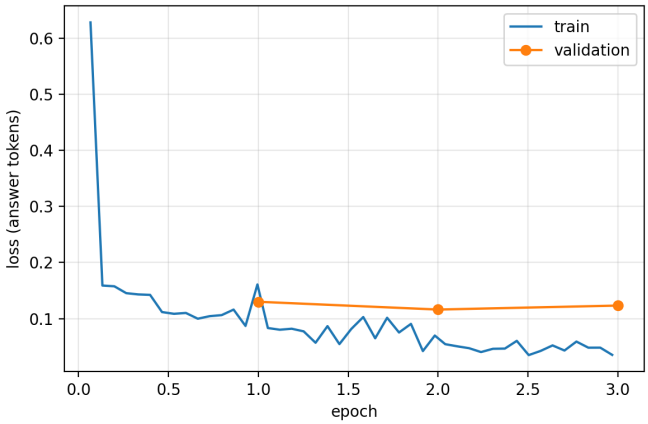


Fig. 2. Training and validation loss (answer tokens). Best checkpoint: epoch 2.

TABLE V
BASE VS FINE-TUNED (FT) RESULTS. EV. = EVIDENCE F1: MICRO / MACRO OVER NON-EMPTY GOLD / MACRO OVER ALL RECORDS.

Split	Model	Acc.	Macro-F1	Ev. micro	Ev. non-empty	Ev. all
Val	Base	0.053	0.017	0.147	0.211	0.315
Val	FT	0.738	0.541	0.520	0.505	0.755
Test	Base	0.073	0.023	0.177	0.265	0.443
Test	FT	**0.718**	**0.517**	**0.489**	**0.457**	0.815

cites a sentence, mirroring the many empty-evidence examples in training.

C. Per-class behaviour

Table VII and Fig. 3 give per-class precision/recall/F1 and confusion matrices on the test split (validation in Appendix A). Three patterns stand out. (1) "Not excluded" has recall 0.99 but precision 0.71 because the model uses it as a default. (2) For exclusion criteria the class "not enough information" has P=R=F1=0 (0 of 28 found): the model never abstains on exclusion. (3) For inclusion criteria the opposite occurs: "not enough information" recall is 0.97, and all 7 "not included" cases are predicted as "not enough information". This is consistent with (but does not prove) the model having learned a per-criterion-type label prior from the training distribution (Table II) instead of reading the evidence. "Included" is the best-behaved decision class (P=0.94, R=0.63), and "not applicable" has high precision (0.80) but low recall (0.27).

TABLE VI
RESULTS BY CRITERION TYPE. EV. = EVIDENCE MICRO-F1 / MACRO-F1 OVER RECORDS WITH NON-EMPTY GOLD EVIDENCE.

Split	Type	Model	Acc.	Macro-F1	Ev. micro	Ev. non-empty
Val	Incl.	Base	0.015	0.007	0.237	0.200
Val	Incl.	FT	0.712	0.367	0.596	0.656
Val	Excl.	Base	0.071	0.035	0.088	0.224
Val	Excl.	FT	0.750	0.501	0.361	0.317
Test	Incl.	Base	0.040	0.019	0.270	0.284
Test	Incl.	FT	0.720	0.384	0.575	0.575
Test	Excl.	Base	0.092	0.043	0.085	0.211
Test	Excl.	FT	0.718	0.433	0.118	0.125

TABLE VII
PER-CLASS RESULTS, FINE-TUNED MODEL, TEST SPLIT. n = SUPPORT.

Scope	Class	P	R	F1	n
Overall	included	0.94	0.63	0.76	27
	not included	0.00	0.00	0.00	7
	not enough information	0.65	0.56	0.60	66
	not excluded	0.71	0.99	0.83	91
	not applicable	0.80	0.27	0.40	15
Inclusion	included	0.94	0.63	0.76	27
	not included	0.00	0.00	0.00	7
	not enough information	0.65	0.97	0.78	38
	not applicable	0.00	0.00	0.00	3
Exclusion	not excluded	0.71	0.99	0.83	91
	not enough information	0.00	0.00	0.00	28
	not applicable	0.80	0.33	0.47	12

D. Error analysis

On test there are 58 label errors (49 involve "not enough information", 9 are applicability confusions) and a further 29 records have the correct label but different evidence IDs. The most frequent expert→model pairs are: not enough information → not excluded (28), included → not enough information (10), not applicable → not excluded (8), not included → not enough information (7). Table VIII analyses eight distinct errors; all twelve sampled errors and our analysis are in the repository.

The errors differ in impact. Missing information treated as negative (category of Error 3) is the dangerous one, because it can make a patient look eligible; for this reason the application adds a caution whenever a "not excluded" prediction has no cited evidence. Applicability confusions (Errors 2, 8, 10) do not change the aggregated outcome. Four of the twelve

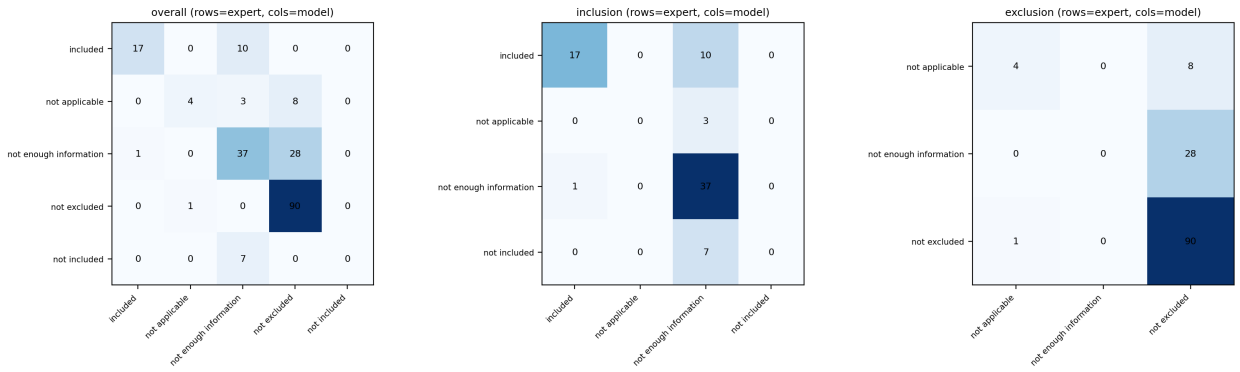


Fig. 3. Confusion matrices on the test split (rows: expert, columns: model): overall, inclusion and exclusion.

TABLE VIII
EIGHT ANALYSED TEST ERRORS (EXPERT → MODEL).

#	Criterion (abridged)	Expert → model	Cause
1	Elective carotid endarterectomy (incl.)	not included → NEI	Over-abstention: the patient had a cardiac catheterization, so the criterion is clearly unmet, but no sentence states it; concluding "not included" needs a negative inferred from another context. The class is rare in training (13/603).
2	Pregnancy / childbearing age without contraception (excl.)	not excluded → not applicable	Label-boundary ambiguity: the patient is a 48-year-old male; the expert cited his sex but chose "not excluded", the model arguably reasonably chose "not applicable".
3	Estimated GFR < 30 (excl.)	NEI → not excluded	**Missing information treated as negative**: no lab value in the note; the model assumed absence means no exclusion (cites nothing). Largest error group (28/58).
5	No history of central venous catheterization (incl.)	included → NEI	Negated criterion with indirect evidence: the only catheterization is arterial (femoral), which needs domain knowledge to distinguish from a central line.
7	If female, using birth control (incl.)	not applicable → NEI	Correct evidence sentence ("no sexual partners") but wrong label; the sentence implies the criterion is irrelevant.
8	Breast-feeding (excl.)	not applicable → not excluded	The patient is a 5-year-old boy; the age was in sentence 0 but the model did not use it and defaulted to the majority label.
9	Sore throat/odynophagia plus $\geq$2 signs (incl.)	NEI → included	Over-confident positive: the criterion text is truncated (the list of signs is missing); dysphagia and drooling were treated as equivalent to the required findings.
10	Alcoholism, drunk at least weekly (excl.)	not applicable → not excluded	Same patient as 8; applicability not recognized. Practical effect is small since both labels are non-blocking in the outcome rule.

sampled errors belong to one patient and three to another, so with only eight test patients a few patients drive many errors.

E. Tiled attention

Table IX compares reference, tiled and PyTorch fused scaled-dot-product attention (batch 1, 8 heads, $d = 64$, fp32, tile 128). The tiled output matches the reference with maximum absolute error $\leq 10^{-6}$ (mean error below 10^{-6}) in every setting, and the unit tests (33 attention cases covering several shapes, block sizes including non-divisors, causal and non-causal masks, $N_q < N_k$, very large score magnitudes and a future-leak check) pass. Peak memory of the reference grows quadratically (420 MB at $N = 2048$) while the tiled version grows only linearly (38 MB, $10.9\times$ less). Runtime, however, is worse: tiled is 1.8–$7.9\times$ slower than the reference and 8–$44\times$ slower than the fused kernel (causal), and it is faster causally than non-causally because future blocks are skipped. This is expected: our implementation launches many small PyTorch operations from a Python loop and writes intermediate tiles to GPU memory, whereas a fused kernel keeps tiles in on-chip SRAM and launches once; the benefit of tiling is memory, not speed, unless it is fused.

F. KV-cached decoding

Table X reports the decoder benchmark (version 2, after fixing a decoding setting, see Section IV-I). In all 15 (configuration, mode) cells the generated tokens are identical to the uncached reference (fp16, greedy). Caching helps more as sequences get longer: $1.06\times$, $3.75\times$ and $6.21\times$ faster than no cache at 128/32, 512/64 and 1024/128 (prompt length / new tokens), because the uncached path recomputes the whole sequence at every step (cost grows roughly quadratically with length) while the cached path processes one token per step. Our hand-written cached decoder is within about $\pm 15\%$ of Hugging Face's `generate` (1.208 vs 1.024 s at 128/32; 4.690 vs 4.524 s at 1024/128). Peak memory is dominated by the weights of both models kept resident ($\approx$3.9 GiB); the differences come from the logits: our decoders let the model compute logits for every prompt position ($1024 \times 151{,}936 \times 2$ bytes $\approx$297 MiB at 1024 tokens) and the uncached path recomputes them at every step, whereas `generate` computes only the last position, which likely explains its lower peak. At 1024/128 the CUDA allocator printed retry warnings for a 330 MiB allocation, which equals the fp16 logits of a $\approx$1,138-

TABLE IX
ATTENTION BENCHMARK ON A TESLA T4 (MS = MEAN OF 5 RUNS AFTER WARM-UP; MB = PEAK ALLOCATED, INCLUDING INPUTS). MAX ERROR OF TILED VS REFERENCE IS $\leq 10^{-6}$ IN ALL ROWS.

N	Mask	Ref. ms	Tiled ms	SDPA ms	Ref. MB	Tiled MB	SDPA MB
128	none	0.278	0.648	0.084	11.1	11.9	9.9
128	causal	0.384	0.686	0.089	11.1	11.9	9.9
512	none	0.770	6.800	0.381	38.1	17.4	15.1
512	causal	0.931	5.878	0.353	38.4	17.4	15.1
1024	none	2.388	27.167	1.548	116.2	24.4	22.1
1024	causal	2.984	23.715	0.878	117.2	24.4	22.1
2048	none	8.641	103.918	3.807	416.2	38.4	36.1
2048	causal	10.463	78.806	1.790	420.2	38.4	36.1

TABLE X
DECODER BENCHMARK, TESLA T4, FP16, MEAN TOTAL TIME OF 5 RUNS, PEAK ALLOCATED MEMORY, TOKEN EQUALITY WITH THE UNCACHED REFERENCE (V2). CONFIG = PROMPT LENGTH/NEW TOKENS.

Config	Mode	Total s	tok/s	Peak MB	Equal
128/32	no cache	1.284	24.9	3999	yes
	KV cache (ours)	1.208	26.5	3948	yes
	speculative (ours)	1.821	17.6	3988	yes
	HF generate	1.024	31.2	3918	yes
	HF assisted	1.978	16.2	3966	yes
512/64	no cache	8.222	7.8	4241	yes
	KV cache (ours)	2.191	29.2	4071	yes
	speculative (ours)	2.404	26.6	4224	yes
	HF generate	2.273	28.2	3954	yes
	HF assisted	1.943	32.9	4018	yes
1024/128	no cache	29.128	4.4	4578	yes
	KV cache (ours)	4.690	27.3	4235	yes
	speculative (ours)	5.033	25.4	4542	yes
	HF generate	4.524	28.3	4002	yes
	HF assisted	4.119	31.1	4041	yes

TABLE XI
PREFILL AND DECODE TIMES (MEAN $\pm$ STD OVER 5 RUNS).

Config	Mode	Prefill s	Decode s
128/32	no_cache	0.04	1.25
	kv_cache	0.04	1.32
	speculative	0.07	1.94
	hf_generate	0.00	1.03
	hf_assisted	0.00	2.22
512/64	no_cache	0.10	8.44
	kv_cache	0.08	2.28
	speculative	0.12	2.52
	hf_generate	0.00	2.47
	hf_assisted	0.00	2.07
1024/128	no_cache	0.20	29.03
	kv_cache	0.19	4.91
	speculative	0.25	5.10
	hf_generate	0.00	4.73
	hf_assisted	0.00	4.49

TABLE XII
SPECULATIVE DECODING STATISTICS ($k = 4$). SPEED-UP IS CACHED TIME DIVIDED BY SPECULATIVE TIME (< 1 MEANS SLOWER).

Config	Acceptance	Tok./target pass	Target passes	Speed-up
128/32	0.488	2.67	12	0.66
512/64	1.000	4.57	14	0.91
1024/128	1.000	4.74	27	0.93

token uncached forward pass; the runs completed. XI

G. Speculative decoding

Speculative decoding produced exactly the same tokens as greedy decoding in every configuration, but it was never faster than plain KV-cached decoding (Tables X and XII): $1.51\times$, $1.10\times$ and $1.07\times$ the time of the cached decoder, i.e. speed-up factors of 0.66, 0.91 and 0.93. The reasons are visible in the acceptance statistics. At 128/32 only 49% of drafted tokens were accepted (2.67 tokens per target pass), so many drafting passes were wasted. At 512/64 and 1024/128 acceptance was 100% (4.57 and 4.74 tokens per target pass), but this is inflated: our benchmark prompts are built by repeating validation notes, so the continuation is highly predictable, and we do not claim it is typical. Even with full acceptance there is no gain here, because a decoding step on this GPU is dominated by per-layer launch overhead rather than weight reads, and the draft has 24 layers against 28 for the target (a draft step costs $\approx$24/28 of a target step). With $k = 4$ a fully accepted round costs five draft passes (four drafts plus one catch-up pass that keeps the draft cache in sync) and one target pass, i.e. about $5\times0.86+1 \approx 5.3$ target-pass equivalents for 5 tokens versus 5 for plain decoding: a predicted $\approx$6% slowdown, against 7–10% observed. This back-of-envelope model is a hypothesis consistent with the data, not a verified profile. A draft that is much cheaper than the target (or a hardware setting that is memory-bound), a draft fine-tuned on the same task, and removing the catch-up pass would be needed for a speed-up. Hugging Face's assisted generation was 11–12% faster than plain cached decoding at 512 and 1024 tokens, which suggests that its implementation overhead (e.g. adaptive draft length and fewer GPU synchronizations) is lower than ours; we did not profile this.

H. Application

Fig. 4 shows a screening run. For the example note the application finds the supporting sentences for the two satis-

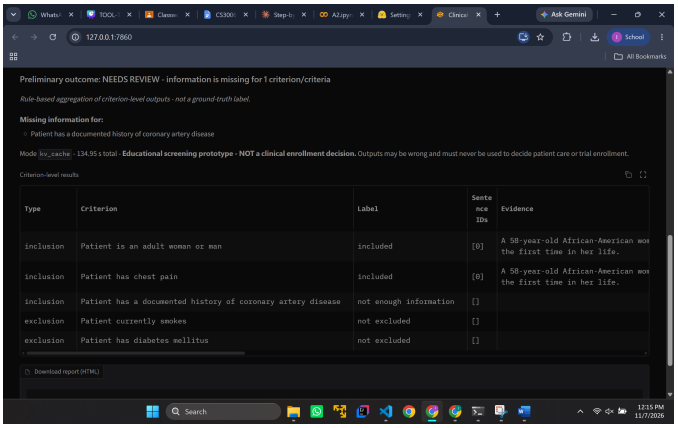


Fig. 4. Screening result in the browser application: criterion, label, cited sentence IDs and evidence text, with the rule-based preliminary outcome.

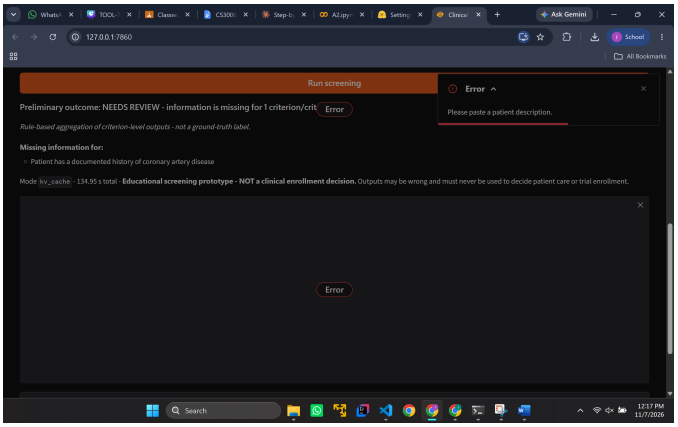


Fig. 5. Input validation: an empty patient description is rejected with a clear message.

fied inclusion criteria, reports "not enough information" for the coronary-artery-disease criterion, and flags it as missing information. For the smoking and diabetes exclusion criteria the model answers "not excluded" but cites no sentence even though sentence 4 states that the patient denies smoking and diabetes: the label is probably right, but the evidence is missing, which is the weakness measured in Table VI. Fig. 5 shows that an empty note produces a readable error instead of a crash.

The three decoding modes produce identical labels and evidence for the same note; the five-criterion example took 200.5 s with speculative and 929.4 s with no-cache decoding on the laptop CPU (same note), and 135.0 s with KV cache on a slightly different note. Figs. 6 and 7 show the benchmark tab. The live comparison (260 prompt tokens, 8 new tokens, CPU) gave 116.7 s (no cache), 16.4 s (KV cache) and 49.7 s (speculative, acceptance 0.83) with identical tokens. These CPU timings are noisy: the prefill of the same prompt took 32.4 s in the first mode, 11.7 s in the second and 32.3 s in the third, which indicates warm-up, power-state or background-load effects, so we do not draw conclusions about speculative

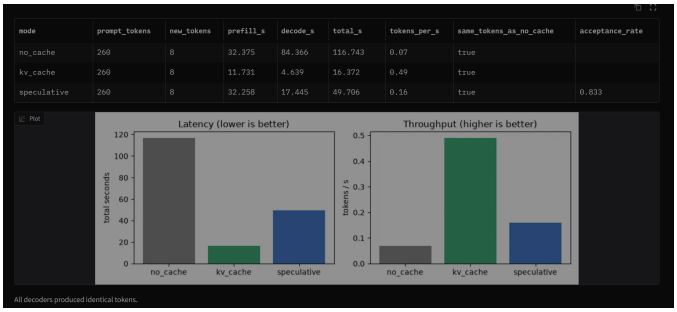


Fig. 6. Live decoder comparison in the application (CPU).

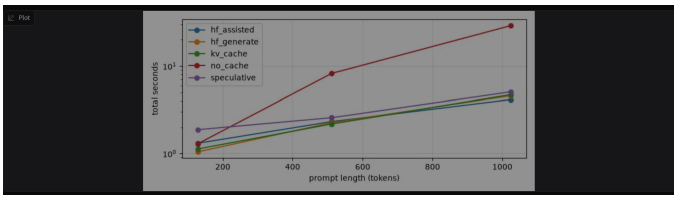


Fig. 7. Logged GPU benchmark loaded in the application: total time versus prompt length for each decoder (log scale).

versus cached decoding from them; the GPU benchmark above is our measurement.

I. Difficulties and design decisions

Prompt without label set. Our first prompt did not list the allowed labels. The base model invented its own labels and all 206 validation predictions failed to parse (accuracy 0). Adding the label set removed all parse failures but accuracy remained 0.053, because the base model predicted "not applicable" for 198 of 206 examples; we kept this as the fair zero-shot baseline and used the identical prompt for training.

Hidden repetition penalty. In the first benchmark, Hugging Face `generate` (plain and assisted) produced different tokens from our greedy paths at 128/32. The library log showed a repetition-penalty processor active even with `do_sample=False`: the model's `generation_config.json` sets a non-default repetition penalty. Setting it to 1.1 in both the evaluation and the benchmark scripts made all modes token-identical (v2). The validation metrics were re-computed with the corrected setting (evidence micro-F1 changed from 0.560 to 0.520 and accuracy was unchanged); test metrics were only computed after the fix. We keep the v1 logs as evidence.

Library incompatibility and cache API. The pre-installed `torchao` on Colab was incompatible with the installed PEFT and was uninstalled; the newer Transformers cache `crop()` expects a negative "tokens to remove" value, which a unit test revealed, and we wrapped it in a helper that works on both API versions.

Rejected options. Full fine-tuning (memory), other model families (gated licenses or tokenizer pairing; not trained for lack of time), class-weighted or oversampled training to improve rare classes (not run; see Section V), and a proprietary API (not allowed).

V. Limitations

- **Small, imbalanced data.** 37 training patients; rare classes ("excluded" 8, "not included" 13, "not applicable" 5 in inclusion) are essentially not learned, and the test split has no "excluded" example and only 7 "not included". With 8 test patients a few patients drive many errors, and we report a single run without confidence intervals.
- **Missing-information bias.** The model never abstains on exclusion criteria and can therefore make a patient look eligible. The application's caution only partly mitigates this. We did not try class re-weighting, oversampling or threshold calibration.
- **Weak evidence for exclusion.** Exclusion evidence F1 is low (0.118 micro on test), partly because the model learned to cite nothing.
- **Explanations are templates** built from the label and cited sentences; the model does not generate free-text reasoning.
- **Limited model comparison.** One model family, one fine-tuning configuration, no hyperparameter search; alternatives were only considered on paper.
- **Efficiency findings are hardware-specific.** Speculative decoding was slower on a T4 with this draft/target pair; acceptance rates come from repeated-text prompts; CPU timings in the application are noisy; peak-memory numbers include both models' weights.
- **Annotation ambiguity.** The "not applicable" versus "not excluded" boundary is applied inconsistently in the gold data (Errors 2 vs 8).
- **Not a clinical tool.** Outputs may be wrong and must not be used for care or enrollment decisions.

VI. Conclusion

A LoRA-tuned 1.5B open model can learn the TrialGPT annotation format and reach 71.8% test accuracy and macro-F1 0.517, a large gain over its zero-shot behaviour, while running locally. Its main weakness, treating missing information as a negative for exclusion criteria, is exactly what a screening aid must avoid, and it is partly a consequence of class imbalance. Our from-scratch KV cache (up to $6.2\times$ faster), tiled attention (exact, linear memory, but slower unfused) and speculative decoder (exact, but slower here) were each verified against references and measured honestly. Future work: class-balanced training and abstention calibration, better evidence supervision for exclusion criteria, a draft model fine-tuned on the task, and a fused tiled-attention kernel.

Appendix A
Validation per-class results

Validation shows the same pattern as test: zero recall for exclusion "not enough information" (0 of 26) and for "not included" (0 of 3).

TABLE XIII
Per-class results, fine-tuned model, validation split.

Scope	Class	P	R	F1	n
Overall	excluded	1.00	0.50	0.67	2
	included	0.83	0.69	0.75	35
	not included	0.00	0.00	0.00	3
	not enough information	0.62	0.43	0.51	53
	not excluded	0.77	0.98	0.86	101
	not applicable	0.50	0.42	0.45	12
Inclusion	included	0.83	0.69	0.75	35
	not included	0.00	0.00	0.00	3
	not enough information	0.62	0.85	0.72	27
	not applicable	0.00	0.00	0.00	1
Exclusion	excluded	1.00	0.50	0.67	2
	not excluded	0.77	0.98	0.86	101
	not enough information	0.00	0.00	0.00	26
	not applicable	0.50	0.45	0.48	11

Appendix B
Reproducibility

- Code: https://github.com/ImamaSarwar/trial-eligibility-llm
- Adapter: https://huggingface.co/ISarwar/trial-lora
- Demo video: https://drive.google.com/file/d/1QKfvqC5c5M_dtheSDrRh3towBYOjUJhY/view?usp=sharing
- Split manifest: `data/split_manifest.json`; logs: `results/`.
- Commands: `python src/data_prep.py`; `python -m src.train_lora`; `python -m src.evaluate --split val|test`; `python -m src.benchmark`; `python -m pytest -q` (51 tests); `python app/app.py`.

Appendix C
AI-Use Statement

Claude was used to plan the project, to help draft the source code, and this report. All code was executed by the author: 51 unit tests pass, and all training, evaluation and benchmark numbers in this paper come from the author's own runs on Google Colab (Tesla T4) and a laptop. The author reviewed the code and the analysis and can explain and modify each component. Existing libraries used: PyTorch, Transformers, PEFT, Datasets, scikit-learn, Matplotlib, Gradio. The TrialGPT repository was consulted as a reference only; no code or predictions were reused. The GPT-4 columns of the dataset were not used.

References

[1] Q. Jin *et al.*, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, art. 9074, 2024.

[2] NCBI, "TrialGPT-Criterion-Annotations," Hugging Face dataset, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations.

[3] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[4] S. Mangrulkar *et al.*, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," 2022. [Online]. Available: https://github.com/huggingface/peft
[5] A. Yang *et al.* (Qwen Team), "Qwen2.5 technical report," arXiv:2412.15115, 2024.
[6] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.
[7] J. Su *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, 2024 (arXiv:2104.09864).
[8] J. Ainslie *et al.*, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, 2023.
[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. NeurIPS*, 2022.
[10] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.
[11] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. ICML*, 2023.
[12] C. Chen *et al.*, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.
[13] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020.
[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.